Building a Production-Ready MLOps AI Agent

Assignment No. 3 — MLOps & Production AI Engineering

Mohammad Hannan Abid
Roll No: 23i-0713
Section: A

Fall 2026

Repository: `https://github.com/HannanAbid7/assignment-03-mlops-agent`

Contents

1 Introduction **3**

2 Complete System Architecture **3**
2.1 Design decisions 3

3 Tool Definitions and Permissions **3**
3.1 System-level safety behaviors 4

4 Evidence from Each System **5**
4.1 DVC 5
4.2 MLflow 5
4.3 Airflow 5
4.4 Kubernetes 5

5 Tested Agent Conversations **6**
5.1 Conversation 1: Mandatory multi-tool integration scenario 6
5.2 Conversation 2: Kubernetes deployment diagnosis (Failure Injection 1) 6
5.3 Conversation 3: Airflow pipeline diagnosis (Failure Injection 2, partial) 7

6 Failure Investigations **7**
6.1 Failure Investigation 1: Kubernetes bad image tag 7
6.2 Failure Investigation 2: Airflow quality gate (plus a real external-API failure) . . 7

7 Testing, Reliability, and Safety **7**

8 GitHub Repository and CI **8**

9 Known Limitations and Proposed Improvements **8**

10 Conclusion **8**

1 Introduction

This report documents the design, implementation, and testing of an AI-powered MLOps Agent capable of understanding natural-language requests and operating four real MLOps systems: DVC (dataset versioning), MLflow (experiment tracking), Apache Airflow (pipeline orchestration), and Kubernetes (deployment management). The agent is built to answer only from real tool evidence, never from its own training knowledge, and to never silently trigger retraining or cluster-state changes.

2 Complete System Architecture

```
User / Engineer (natural language)
        |
        v
FastAPI /chat endpoint (agent/main.py)
        |
        v
Orchestrator (agent/orchestrator.py)
  - sends system prompt + 21 tool schemas + conversation to an LLM
    (Groq, OpenAI-compatible API, model openai/gpt-oss-120b)
  - LLM decides which tool(s) to call
  - orchestrator executes the real tool function
  - result (status/data/error) fed back to the LLM
  - loop continues until a final answer (max 16 iterations,
    retries on HTTP 429 with real backoff)
        |
   -----+-----+-----+-----
   |    |     |     |
   v    v     v     v
  DVC  MLflow Airflow Kubernetes
```

2.1 Design decisions

- **No agent framework (e.g. LangChain) was used.** Tool calling is implemented directly against the OpenAI-compatible chat-completions API, so every tool invocation is explicit, inspectable in logs, and easy to reason about for the safety requirements below.
- **Tool schemas are auto-generated** from each Python function's real signature via `inspect.signature`, so the JSON schema sent to the LLM can never drift out of sync with the actual implementation (parameter names, types, and required/optional status are always consistent).
- **Every tool returns a uniform envelope**: {`"status": "ok"|"error", "data": ..., "error": ...`}, so the orchestrator can report failures honestly to the LLM instead of crashing or fabricating a result.

3 Tool Definitions and Permissions

Each of the four modules exposes 5–6 callable tools (21 total), satisfying the "at least three tools per module" requirement. Table 1 lists every tool with its real docstring (used verbatim as the tool description sent to the LLM) and its safety/permission guard.

Module	Tool	Permission / Safety Guard
DVC	`list_tracked_datasets`	read-only
DVC	`get_current_version`	read-only
DVC	`compare_versions`	read-only
DVC	`has_data_changed`	read-only
DVC	`get_reproduction_instructions`	read-only, no subprocess call
DVC (internal)	`_run_dvc`	subprocess restricted to an allow-list: `status`, `list`, `diff`, `checkout` only — no destructive DVC command is reachable
MLflow	`list_experiments`	read-only
MLflow	`list_runs`	read-only
MLflow	`get_run_details`	read-only
MLflow	`get_best_run`	read-only; always returns `production_ready: false` with an explicit caveat so "highest metric" is never conflated with "safe to deploy"
MLflow	`compare_runs`	read-only
MLflow	`get_model_versions`	read-only
Airflow	`list_dags`	read-only; filtered to this project's own DAGs (excludes ∼115 Airflow-bundled example DAGs)
Airflow	`get_dag_run_status`	read-only
Airflow	`trigger_dag`	**state-changing** — allow-listed to `training_pipeline` only; any other `dag_id` raises `PermissionError`
Airflow	`get_failed_tasks`	read-only
Airflow	`get_task_logs`	read-only
Kubernetes	`list_pods_and_deployments`	read-only; restricted to `mlops-agent` namespace
Kubernetes	`get_pod_status_and_logs`	read-only; namespace-restricted
Kubernetes	`inspect_services`	read-only; namespace-restricted
Kubernetes	`get_unhealthy_pods`	read-only; namespace-restricted
Kubernetes	`get_deployed_version`	read-only; namespace-restricted
Kubernetes	`restart_deployment`	**state-changing** — requires explicit `confirm=True` *and* the approved namespace; silently refuses otherwise

Table 1: Complete tool inventory and permission model

3.1 System-level safety behaviors

1. The system prompt instructs the model to only state facts backed by a tool result, and to explicitly say when a tool errored or a service was unavailable, rather than guessing.

2. Retraining and cluster-state changes are never triggered automatically: the system prompt instructs the model to only *recommend* such actions, and the only two state-changing tools (`trigger_dag`, `restart_deployment`) carry their own independent guards regardless of what the model decides.

3. Every tool call is logged with a request ID, tool name, status, and duration — no secrets

are ever logged.

4. Malformed tool-call arguments from the LLM are caught and reported as a structured error instead of crashing the request.

4 Evidence from Each System

4.1 DVC

```
$ dvc status
Data and pipelines are up to date.

$ git tag -n
dataset-v1 fix: properly track dataset with DVC instead of git, ...
dataset-v2 chore: update dataset to v2
```

Two real dataset versions were created and tagged during development: training on `dataset-v1` produced runs with F1 ≈ 1.0; after removing the last 10 rows of the Iris dataset to produce `dataset-v2`, F1 dropped to ≈ 0.93 — a genuine, observable effect of dataset versioning on model quality, which the agent's `get_current_version` and `compare_versions` tools can detect.

4.2 MLflow

```
run_id params metrics
f4ab478858bb n_estimators=100, max_depth=None acc=0.929, f1=0.930
efe363d914a0 n_estimators=100, max_depth=None acc=0.929, f1=0.930
2d3248b202ac n_estimators=200, max_depth=10 acc=1.000, f1=1.000
dac909699dcf n_estimators=150, max_depth=5 acc=1.000, f1=1.000
```

Eight manual training runs plus three Airflow-triggered runs were logged to the `iris-training` experiment across both dataset versions, each tagged with its producing git commit and DVC data hash.

4.3 Airflow

```
$ GET /api/v2/dags/training_pipeline/dagRuns
manual__2026-11-11T18:08:43 -> success (threshold 0.6, normal run)
manual__2026-11-11T18:49:54 -> success (threshold 0.99, best historical
                                        run still cleared the gate)
manual__2026-11-11T18:50:31 -> failed (threshold 1.5, quality gate
                                       correctly blocked promotion)
```

The DAG (`validate_dataset` → `preprocess_data` → `train_model` → `evaluate_and_log_mlflow` → `quality_gate`) was triggered three times via the real Airflow REST API (token-authenticated), demonstrating both a successful run and a correctly-blocked run (Failure Injection 2, Section 6).

4.4 Kubernetes

```
$ kubectl get pods -n mlops-agent
NAME READY STATUS RESTARTS AGE
iris-api-644d56d745-4kh6c 1/1 Running 0 157m
iris-api-644d56d745-k4tr2 1/1 Running 0 157m

$ kubectl get svc -n mlops-agent
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
iris-api ClusterIP 10.96.129.107 <none> 80/TCP
```

```
$ kubectl get deployment -n mlops-agent
NAME READY UP-TO-DATE AVAILABLE AGE
iris-api 2/2 2 2 157m
```

The inference service runs as a 2-replica Deployment with readiness/liveness probes on `/health`, CPU/memory requests and limits, and configuration via a ConfigMap, fronted by a ClusterIP Service — all required manifest elements are present (`kubernetes/*.yaml`).

5 Tested Agent Conversations

Three full conversations were run against the live agent (real LLM calls, real tool execution against the running DVC/MLflow/Airflow/Kubernetes stack). A fourth and fifth conversation were planned but could not be completed with full final-answer text because the project's free-tier LLM API key exhausted its daily token quota partway through testing — this is documented honestly as Known Limitation 1 (Section 9) rather than fabricated.

5.1 Conversation 1: Mandatory multi-tool integration scenario

Prompt: *"Find the best recent model, identify the dataset version used to train it, check the deployment status, and explain whether retraining is necessary."*

Tools called (in order): `get_best_run` → `list_pods_and_deployments` → `get_deployed_version` → `compare_versions` → `get_model_versions` → `list_tracked_datasets` → `get_current_version`.

Agent's final answer (abridged):

> **Known facts** — Best recent run `2d3248b2...` (accuracy = 1.0, f1_score = 1.0), tagged with `dataset_version=v1` and DVC hash `4d301ab...`. Current dataset hash `c49abde...` differs from the run's hash — the data has changed since that training. Deployment `iris-api` is running version `iris-api:1.0` with 2 healthy pods. Model registry is empty (no registered versions).
>
> **Uncertainties** — The deployed version has no explicit link to an MLflow run ID; cannot confirm which model is actually serving.
>
> **Recommendations** — Register the validated model in the MLflow Registry with its dataset hash and commit; since the current dataset differs from the one used to train the best run, retraining on the latest data is advisable — but this is a *recommendation only*, not an automatic action.

This demonstrates correlation of evidence across all four systems and correctly stops short of auto-triggering retraining, per the assignment's mandatory requirement.

5.2 Conversation 2: Kubernetes deployment diagnosis (Failure Injection 1)

Prompt: *"Why is my prediction API unavailable?"*

Injected failure: `kubectl set image deployment/iris-api iris-api=iris-api:bad -n mlops-agent`, producing a third pod stuck in `ErrImagePull` alongside two healthy pods.

Agent's final answer (abridged):

> **Known facts** — Two pods (`...4kh6c`, `...k4tr2`) are `Running` with logs showing successful `GET /health` calls. A third pod (`...5pnzc`) is `Pending`: "trying and failing to pull image" — a container-image-pull failure.
>
> **Recommendations** — Inspect the failing pod's events (`kubectl describe pod`), validate the image reference in the Deployment spec, fix the tag/registry issue, and roll out again; consider temporarily scaling to the two healthy replicas while the image is fixed.

The agent correctly distinguished healthy from unhealthy pods using real pod status and logs, rather than guessing at the cause.

5.3 Conversation 3: Airflow pipeline diagnosis (Failure Injection 2, partial)

Prompt: *"The last training pipeline run seems to have failed, what happened and why?"*

Injected failure: `QUALITY_THRESHOLD` raised from 0.6 to 1.5 (unreachable), DAG triggered.

Tool chain executed (verified in logs): `list_dags` → `get_dag_run_status` → `get_failed_tasks` → `get_task_logs`. The agent correctly identified that all four upstream tasks succeeded and `quality_gate` was the failed task, fetching its logs as evidence.

Outcome: the LLM's final natural-language synthesis was cut off mid-generation because the project's Groq API key hit its 200,000 token/day cap during this exact test (see Limitation 9). The tool-selection and evidence-gathering logic itself is verified correct from the structured logs; only the very last synthesis step did not complete. This is reported honestly rather than fabricated, and is itself documented as Failure Investigation 2.

6 Failure Investigations

6.1 Failure Investigation 1: Kubernetes bad image tag

Root cause: Deployment updated to reference a non-existent image tag (`iris-api:bad`).
Resolution: `kubectl rollout undo deployment/iris-api -n mlops-agent`; pods returned to `2/2 Running` within seconds.

6.2 Failure Investigation 2: Airflow quality gate (plus a real external-API failure)

Root cause (intended): `QUALITY_THRESHOLD` set above any achievable F1 score, so the quality gate correctly blocked promotion of an otherwise-successfully-trained model.
Root cause (unintended, discovered during this investigation): the Groq free-tier API key's *daily* token quota (200,000 TPD) was exhausted mid-test, which is a genuine infrastructure limitation of the chosen LLM provider, separate from the agent's own logic.
Resolution: `QUALITY_THRESHOLD` reverted to `0.6`; the daily-quota issue is documented as a known limitation (upgrading to a paid tier or adding proactive token-budget tracking are the proposed fixes).

7 Testing, Reliability, and Safety

Ten automated `pytest` tests cover all four tool modules plus the orchestrator, with every external service mocked:

- `test_rejects_non_allowlisted_subcommand` — DVC command allow-list enforced.
- `test_get_best_run_picks_highest_metric`, `test_get_best_run_no_matches` — MLflow tool correctness.
- `test_trigger_dag_rejects_unapproved` — Airflow DAG allow-list enforced.
- `test_get_dag_run_status_timeout` — Airflow tool handles a connection timeout without crashing.
- `test_rejects_wrong_namespace` — Kubernetes namespace restriction enforced.
- `test_restart_requires_confirm` — unauthorized cluster-state change rejected.
- `test_no_invented_results_on_tool_failure` — orchestrator reports a tool failure as an error, never a fabricated result.

- `test_tool_success_path` — successful tool execution and result propagation.
- `test_logs_contain_no_secrets` — structured logs never expose credentials.

All 10 tests pass (`pytest -v`, verified locally and in CI via GitHub Actions on every push and pull request).

Two deliberately injected failures were demonstrated end-to-end (Section 6), satisfying the requirement to show at least two and explain how the agent diagnoses them.

8 GitHub Repository and CI

The repository follows the assignment's recommended structure (`agent/`, `api/`, `dags/`, `kubernetes/`, `tests/`, `.github/workflows/`). A GitHub Actions workflow (`.github/workflows/ci.yml`) runs linting and the full pytest suite on every push and pull request, with all external service calls mocked so CI requires no live infrastructure.

9 Known Limitations and Proposed Improvements

1. **Third-party API rate limits.** The free Groq tier enforces an 8,000 token/minute and 200,000 token/day cap. Long multi-tool conversations (especially ones pulling Airflow task logs) can exceed this, causing slow responses while the orchestrator backs off and retries, or — once the *daily* cap is hit — a hard failure that cannot be retried around (observed directly in Conversation 3 / Failure Investigation 2). **Improvement:** move to a paid tier or self-hosted model, or add a proactive token-budget tracker.

2. **Tool-result truncation can hide information.** Tool output is capped at 1,500 characters before being sent back to the LLM to control token usage; this can cut off relevant content in very large responses. **Improvement:** summarize large payloads server-side instead of blind truncation.

3. **The training DAG's quality gate evaluates the best historical MLflow run, not strictly the run just produced**, since runs are not filtered by the triggering Airflow run ID. This means the gate can pass even if the *current* training run underperforms a past one. **Improvement:** tag each MLflow run with its Airflow `run_id` and filter the quality gate on that specific run.

4. **No formal model registry workflow.** Models are logged to MLflow runs but never registered/promoted automatically — this is intentional (the assignment requires that promotion never happen automatically), but it means the agent cannot yet answer "what model is in production" from the registry alone.

5. **Single-node demo topology.** Airflow runs via `airflow standalone` and Kubernetes runs on a local `kind` cluster — appropriate for this assignment, not representative of a production deployment.

10 Conclusion

The MLOps AI Agent successfully integrates DVC, MLflow, Airflow, and Kubernetes behind a single conversational interface, answers only from real tool evidence, correctly withholds automatic retraining/cluster changes behind explicit safety guards, and was validated against two deliberately injected failures plus a mandatory multi-tool integration scenario. The one incomplete test run was caused by an external API quota limit rather than a flaw in the agent's own

reasoning or tool-selection logic, and is documented transparently above as both a limitation and a real-world engineering lesson about depending on third-party rate-limited services.